Cab Booking Platform— Python Stack

A full-stack ride-hailing application with a **Python backend** (FastAPI) and a React/Next.js frontend (UI using Tailwind). The backend handles auth verification, ride matching, real-time updates, payment webhooks, and background jobs. The system is designed to be deployable (Docker), scalable (Postgres + Redis + Celery), and production-ready.

Primary responsibilities

- Frontend: Next.js + React + Tailwind (keeps modern UI / SSR benefits)
- Backend: **FastAPI (Python)** — REST + WebSocket endpoints
- Auth: Clerk (frontend) or Supabase Auth with Python verification — tokens verified in FastAPI
- Database: **PostgreSQL** (primary), with PostGIS extension for geospatial queries (nearby drivers). Optionally MongoDB for flexible documents.
- Real-time: **python-socketio** (or FastAPI WebSockets) backed by Redis for message brokering
- Background tasks: **Celery + Redis** (or RQ) for sending receipts, heavy jobs, reconciliation
- Payments: **Stripe (stripe-python)** with server-side webhooks
- Geolocation & Maps: Google Maps JS on frontend + **google-maps-services-python** on backend for distance/time/fare calc
- Caching/Store: **Redis** for ephemeral driver locations, rate-limiting, and presence

---

2. Key Features (mapped to Python tech)

- **User Authentication**: Clerk or Supabase Auth on frontend; FastAPI middleware verifies tokens. Role-based access (rider/driver) stored in DB.

- **Ride Booking System**: FastAPI endpoints to create ride requests, estimate fare (distance/time via Google Maps APIs), store requests in Postgres.

- **Driver Dashboard(Optional)** : Driver connects via Socket.IO; server pushes ride requests; drivers accept/decline.

- **Google Maps Integration**: Frontend uses Google Maps JS; backend uses Google Maps Python client for distance matrix/directions.

- **Real-Time Updates**: python-socketio or FastAPI WebSockets + Redis pub/sub to push driver location, ETA, status changes.

- **Payment Gateway**: stripe-python to create PaymentIntents; FastAPI webhook endpoint to confirm payments and update ride status.

- **Ride History & Receipts**: Rides stored in Postgres; Celery sends receipt emails and generates PDF receipts (wkhtmltopdf or WeasyPrint).

- **Ratings & Reviews**: Simple normalized tables; FastAPI endpoints to post/get reviews with aggregation queries.

---

3. Suggested Architecture (high-level)

1. **Next.js (frontend)** ⟷ 2. **FastAPI (backend REST + WS)** ⟷ 3. **Postgres + PostGIS**, **Redis**, **Celery workers**, **Stripe**, **Google Maps APIs**

- Deploy frontend on Vercel; backend on a container host (e.g., Render, Railway, AWS ECS, DigitalOcean App), Postgres managed (Supabase/Postgres), Redis (managed), Celery workers in separate containers.

---

4. Data Models (short)

- **users**: id, name, email, phone, role (rider/driver), profile_pic, rating_avg
- **drivers**: user_id FK, vehicle_info, verified(boolean), current_location (lon/lat point), status (active/offline)
- **rides**: id, rider_id, driver_id, pickup_point (geom), drop_point (geom), status, fare_estimate, fare_actual, distance_meters, duration_secs, created_at
- **payments**: id, ride_id, stripe_payment_intent_id, amount, status
- **reviews**: id, ride_id, rater_id, rated_id, rating, comment

Use PostGIS types for pickup/drop and create geospatial indices for fast nearby-driver queries.

---

5. Example API endpoints (FastAPI)

- `POST /api/auth/session` — verify token from Clerk/Supabase
- `POST /api/rides/request` — create ride request (returns ride id)
- `GET /api/rides/estimate?origin=...&dest=...` — return fare estimate (calls Google Distance Matrix)
- `GET /api/rides/{id}/status` — poll or subscribe via ws
- `POST /api/webhooks/stripe` — Stripe webhook for payment events
- `ws /ws/driver/{driver_id}` — driver socket to receive requests and send location
- `ws /ws/rider/{rider_id}` — rider socket to watch driver

---

6. Two-week Python-centric schedule (adapted from your original plan)

Week 1: Core Backend & Integration

- **Day 1 — Project init & auth**
    - Initialize monorepo: `frontend/` (Next.js) and `backend/` (FastAPI)
    - FastAPI project scaffold (use `uvicorn`, `poetry` or `pipenv`/venv)
    - Integrate Clerk or Supabase auth on frontend; implement token verification in FastAPI middleware
- **Day 2 — Database & models**
    - Set up Postgres (local or Supabase) + PostGIS extension
    - Define SQLAlchemy/SQLModel models and Alembic migrations
- **Day 3 — Ride request & estimate**
    - Implement `/estimate` using google-maps-services-python
    - Implement `/rides/request` to write ride to DB
- **Day 4 — Driver dashboard & matching(Optional)**
    - Implement nearby-driver query (PostGIS) and simple matching logic
    - Expose endpoints for driver availability
- **Day 5 — Realtime basics (Socket.IO)(Optional)**
    - Integrate python-socketio or FastAPI WebSockets + Redis message queue
    - Basic driver-rider connection & location updates
- **Day 6 — Payments (Stripe)**
    - Create PaymentIntent using stripe-python

    - Implement `POST /webhooks/stripe` to handle confirmation

- **Day 7 — Buffer & tests**
    - Unit tests for API endpoints; manual verification of flows

Week 2: Features, Hardening & Deployment

- **Day 8 — Ride history & receipts**
    - Implement ride history endpoints; generate PDF receipt with WeasyPrint
    - Configure Celery task to email receipts
- **Day 9 — Ratings & Reviews**
    - Implement review endpoints + aggregation
- **Day 10 — Promo codes, wallet**
    - Simple promo table and application logic; add wallet balance field and top-up via Stripe
- **Day 11 — UI enhancements**
    - Polish Next.js UI, accessibility, mobile responsiveness
- **Day 12 — Security & performance**
    - Rate limiting (Redis), secure CORS, input validation, DB indices, optimize queries
- **Day 13 — Final testing**
    - End-to-end testing, simulate multiple drivers and rides
- **Day 14 — Dockerize & deploy**
    - Dockerfiles, `docker-compose` for local dev (FastAPI, Postgres, Redis, Celery), deploy to chosen platform

---

7. Implementation Notes & Tips

- **Geospatial queries(Optional)**: Use PostGIS `ST_DistanceSphere` or `ST_DWithin` for fast nearest-driver search; store driver location as `geometry(Point, 4326)`
- **Real-time scale**: Use Redis pub/sub and multiple Socket.IO workers behind a load balancer. python-socketio supports Redis manager for event broadcast.
- **Background tasks**: Use Celery for sending receipts, calling slow third-party APIs, cleanup tasks.
- **Testing**: Use pytest + HTTPX AsyncClient for FastAPI route tests. Use factories (FactoryBoy) for DB fixtures.
- **Security**: Verify auth tokens server-side; protect webhook endpoints using stripe signatures; never expose secret keys.

Resources (Python-focused)

A curated list of official docs, tutorials, libraries, and example repos to accelerate development.

Official docs (must-read)

- FastAPI — https://fastapi.tiangolo.com/
- Stripe Python SDK & guides — https://docs.stripe.com/sdks
- Google Maps Web Services (Python client) — https://developers.google.com/maps/web-services/client-library
- python-socketio — https://python-socketio.readthedocs.io/
- Celery (Redis as broker) — https://docs.celeryq.dev/
- Supabase Python auth reference — https://supabase.com/docs/reference/python/auth-signinwithoauth

- **Using Leaflet.js (Recommended for MVP)**
  Leaflet crash course | All you need to know about leaflet | Leaflet | Tekson

Libraries & tools (Python)

- **SQLModel / SQLAlchemy / Alembic** — models & migrations (choose SQLModel for close FastAPI integration)
- **psycopg2 / asyncpg** — Postgres drivers
- **python-socketio** — realtime Socket.IO server
- **stripe** (stripe-python) — payments
- **googlemaps** — Google Maps Services Python client
- **Celery + redis** — background jobs
- **WeasyPrint / wkhtmltopdf** — PDF receipts
- **pytest + HTTPX** — testing FastAPI APIs

Tutorials & guides

- Real Python — Getting started with FastAPI. Example: "Get started with FastAPI".
- FastAPI official tutorial (interactive docs + examples).
- Tutorials on integrating Stripe with Python + webhooks (Stripe docs).

Example/open-source repos (to study)

- FastAPI + Postgres + Redis ride-sharing samples: search for `uber-clone` / `ride-sharing` on GitHub (many examples built with FastAPI).
- `parthasarathydNU/uber-ride-api` — Uber-like API simulation.

- `googlemaps/google-maps-services-python` — official python client for Google Maps.

Quick dev setup (local) — minimal

```
# create backend venv
python -m venv env
source env/bin/activate
pip install fastapi uvicorn sqlmodel asyncpg alembic python-socketio[asyncio] redis celery stripe googlemaps

# run dev server
uvicorn backend.main:app --reload
```

Tech Stack (FastAPI / Flask + React)

Frontend

- **React.js** (SPA framework)
- **Next.js** (optional for SSR & routing)
- **Tailwind CSS** (UI styling)
- **React Query / SWR** (data fetching)
- **Map Libraries**: Google Maps JavaScript API, react-google-maps/api
- **State Management**: Redux Toolkit or Recoil

Backend (Choose one)

Option 1 — FastAPI (Recommended)

- FastAPI (core backend framework)
- Uvicorn / Hypercorn (ASGI server)
- SQLAlchemy + Alembic (ORM + migrations)
- PostgreSQL / Supabase (database)
- Redis (caching + real-time pub/sub)
- python-socketio (real-time ride tracking)
- Stripe Python SDK (payments)
- python-maps (Google Maps Directions client)

Option 2 — Flask

- Flask (core backend framework)
- Flask-SocketIO (real-time updates)
- SQLAlchemy
- PostgreSQL / Supabase
- Redis
- St